

StatCentral: The Universal Statistics Tracking Platform for Sports Fans and Leagues

Section I: Functional View of the System

This system has two types of users (as shown in the [Figure 1](#) below):

1. League
2. Fan

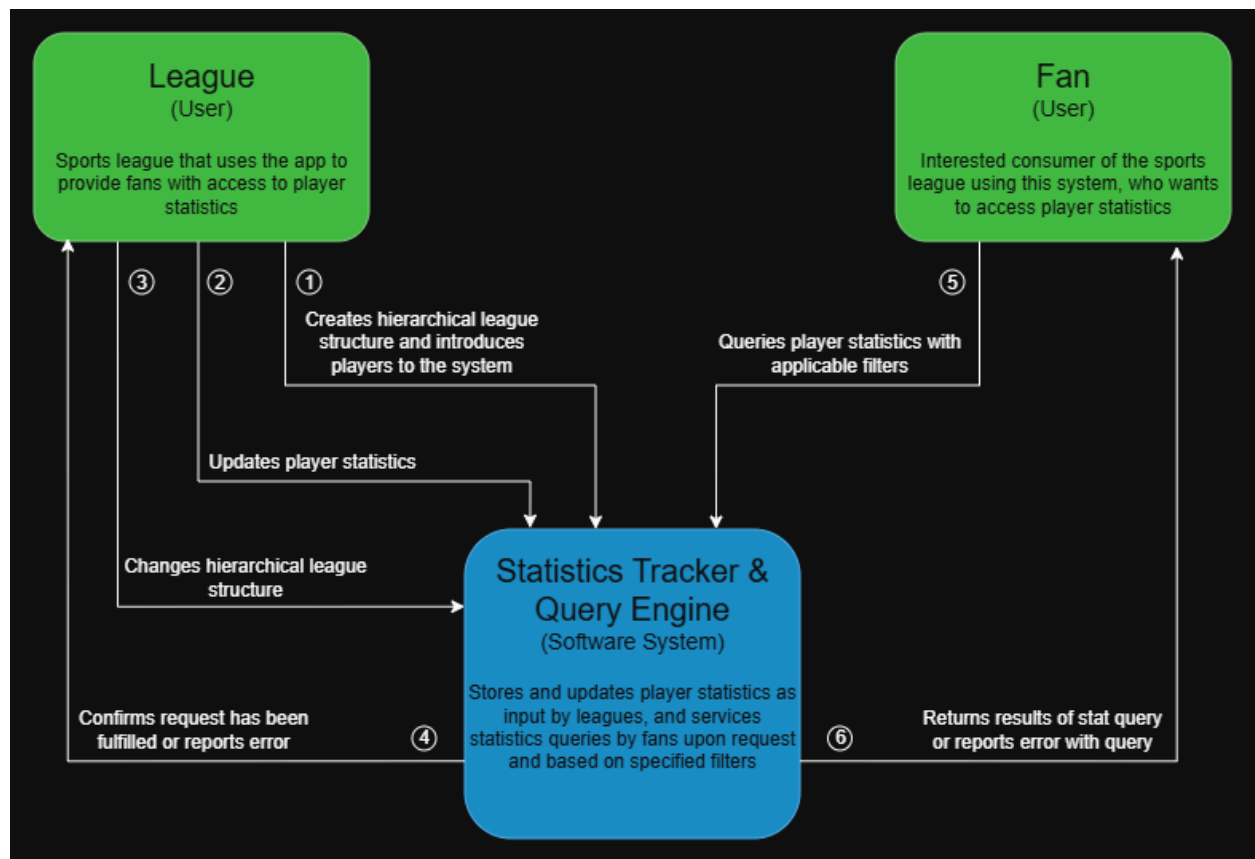


Figure 1

For all leagues that use the system, there is a mechanism by which the league can input player profiles (Figure 1, arrow 1). However, depending on which league is using the system, the engineer of the system will have defined an API tailored to the statistics applicable to that league.

Let's say, for example, MLB (Major League Baseball) is the user. The engineer behind the system will define functions that MLB can call to update player statistics (Figure 1, arrow 2) like batting average, strikeouts, and home runs. It's important to note that certain statistical categories may be dependent on others (e.g. every time a player gets a "hit," their "batting average" increases). This will be accounted for and calculated in the engineer-defined API for the given sport.

In addition to all of their statistics, player profiles will have other fields by which fans can search, such as their position and the team they play for. This is especially important so that fans may filter players by these categories.

This system is designed not only for leagues, but for the practical use of the other user of this system: the fans. From the casual fan who wants to know who the best players are in certain common statistical categories, to the more hard-core ones who want to dive deeply into comparisons of all sorts. As explained earlier, the system caters to the desires of the fans by granting them the ability to query (Figure 1, arrow 5) all the statistics they'd like to analyze, as well as the ability to filter based on statistical categories, league structure (explained below), and other relevant player fields.

A basic (oversimplified) use-case of the system is as follows.

The system has been built to cater toward MLB, and as such includes the following stats (as aforementioned): batting average, strikeouts, and home runs. The system also has a framework whereby MLB can define the names and structure of conferences, divisions, and teams (e.g. MLB creates two conferences defined as "AL" and "NL". Within "AL" there is a division named "East" and within "East" there are a number of teams whose names are decided by MLB). MLB will create player profiles, and for each player, input values for system-required fields (e.g. team, position, etc.).

When a fan approaches the system and its version built for MLB, it can make all sorts of queries. It can query the names of every player in all of MLB, and the system will return to it exactly that (Figure 1, arrow 6). If the fan decides they want to narrow down their search to the top ten players with the most strikeouts in the "East" division of the "AL" conference, the system will filter out all players in all other divisions, and return the top ten in the "strikeouts" category among non-filtered-out players.

In the case of a league using this platform, a failure scenario could occur as follows. Suppose MLB had defined a proper hierarchical structure of conferences, divisions, and teams (Figure 1, arrow 1). If the league then attempted to create a player profile whose "team" field was set to the name of a team that had not yet been defined, the system would return an error message (Figure 1, arrow 4) explaining that the user must assign player profiles to defined teams. An error message would similarly be returned to the user if, for example, the league attempted to define a player profile with an existing "conference" but a "division" that does not exist within the

defined conference in the league structure (e.g. player with “conference” == “AL” but “division” == “Hufflepuff”).

On the fans’ side, the following failure scenario could occur. If, for example, a fan filtered for the top player in a statistical category that has not yet been defined by the system (e.g. “RBI”), an error message would be returned to the user (Figure 1, arrow 6) reflecting the lack of such a category. The same would apply in scenarios where the fan filters among a “division” or a “position” that does not exist within the system.

In addition to failure scenarios surrounding input validation, the system is also built to handle failures within the system itself. As will be explained in the “System Design - High Level” section of this paper, failures among different parts of the system will yield different results. In certain scenarios, where critical parts of the system have crashed or are unreachable, the user will simply receive an error message in response to their request. In other cases, the system may be able to overcome internal failures on its own, and fulfill the request regardless.

A final and quite important feature of this system is its extensibility. Any of the aforementioned errors could potentially occur within the system, whether as a result of a request by a league user or a fan user, because of information that has not been defined by the league at the time of the request. However, just because the league has not yet defined certain criteria, doesn’t mean it can’t do so at any given time. In real life, a sports league may, for example, add a team, re-arrange its hierarchical structure, or introduce new statistical categories. As such, the system is configured to account for the possibility that leagues may want to edit their platform, even after it initially goes public to the fans.

To put this into practice, take the aforementioned scenario where the fan queries the “RBI” statistical category and receives an error message saying that category has not been defined by the league. If the league decided later to ask the system engineer to introduce the RBI stat to the platform, the fan could then successfully query that stat.

If we wanted to use an example where the league has direct control to change aspects of the system (without the aid of the system engineer to expand the API), we could focus on the scenario where the league wants to define a player with a nonexistent “division.” All the league would have to do in order for this request to succeed would be to first create and define that division (e.g. league creates “division” with name “Hufflepuff” under the “conference” titled “AL”). Once the league has altered its hierarchical structure in the system (Figure 1, arrow 3), it can successfully create the player profile with this “division” field.

Section II: System Design - High Level

The system is a distributed statistics management platform designed to allow league administrators to manage players and league structure while allowing fans to query and analyze player statistics. The system is organized as a layered architecture in which client-facing interfaces communicate with backend servers responsible for routing, processing, and executing requests against an underlying PostgreSQL database.

Two primary types of users interact with the system: league users and fan users. League users are responsible for managing players, league structure, and statistical updates, while fan users retrieve and analyze player statistics. Each type of user interacts with a specialized client interface, which communicates with the system through a centralized gateway server. Requests are then routed through a series of backend components until they reach the appropriate microservice responsible for performing the requested operation. Once the operation is completed (or an error surfaces), the response is returned back through the same chain of components to the originating client.

The complete high-level architecture of the system and the way users interact with it is as shown below in [Figure 2](#), and as explained below.

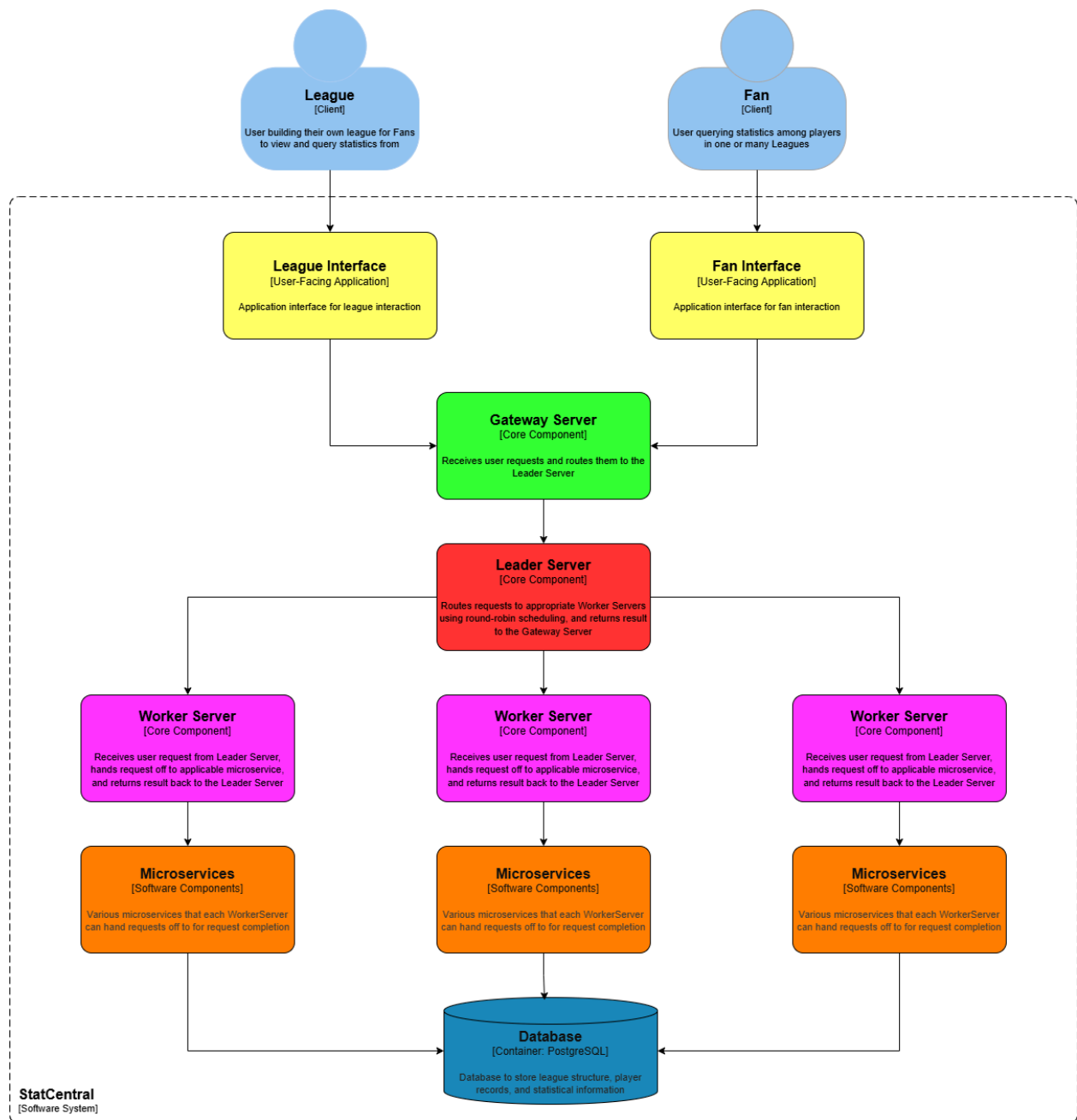


Figure 2

(Note: this figure depicts order of operations among system components from user to database when a request is made. When the request is fulfilled, the result is propagated from database to user in the opposite direction i.e. arrows flipped)

Client Interfaces

The system exposes two application interfaces: the LeagueInterface and the FanInterface.

The LeagueInterface allows league administrators to perform management operations on the system. These operations include creating and deleting player records, modifying the league hierarchy, and updating player statistics. Structural modifications can include moving players between teams, moving teams between divisions, or moving divisions between conferences. Statistical updates allow administrators to increment or directly set a player's statistical values.

The FanInterface provides functionality for users who want to retrieve player statistics. Fans can request the statistics of a particular player or run statistical queries that aggregate information across the league structure. For example, a fan might request the top ten players in a particular conference for a specific statistical category in a given year.

Both interfaces operate in the same way from a networking perspective. Each method invoked on the interface asynchronously sends an HTTP request to the system's GatewayServer. After the request is processed by the backend system, the interface returns the HTTP response received from the gateway to the user.

Gateway Server

The GatewayServer serves as the entry point to the backend of the system. All requests originating from the LeagueInterface or FanInterface are first received by this component via HTTP.

Upon receiving a request, the GatewayServer performs minimal processing and forwards the request asynchronously to the LeaderServer using TCP communication. The gateway acts primarily as a network boundary between external clients and the internal distributed system.

After the backend processing of a request is completed, the GatewayServer receives the result from the LeaderServer and returns it to the originating interface as an HTTP response. This ensures that every HTTP request issued by a client receives a corresponding HTTP response.

The main purpose of the GatewayServer as part of the system is in order to provide an abstraction between the user interface layer and the backend cluster. As is depicted in Figure 2 (above) and will be explained in further detail below, the LeaderServer is responsible for the allocation of requests among WorkerServers. In the event that the LeaderServer crashes, a new one will be elected by the surviving WorkerServers in the cluster.

Given that the user interface layer is not privy to the results of such an election, there must be a component in the system that knows which machine has been elected as the new leader and can ensure that requests are forwarded to it. With the GatewayServer in place, the user interface can be assured that its requests will be forwarded to the leader of the cluster regardless of unexpected crashes and changes to backend cluster hierarchy.

Leader Server

The LeaderServer is responsible for coordinating request distribution within the cluster of backend machines.

When a request arrives from the GatewayServer, the LeaderServer selects the next WorkerServer to handle the request using a round-robin scheduling strategy. Each incoming request is assigned to the next WorkerServer in a rotating sequence, distributing the workload evenly across all available workers over time.

Once the appropriate worker has been selected, the LeaderServer forwards the request asynchronously to that WorkerServer via TCP. After the worker finishes processing the request and returns the result, the LeaderServer forwards that result back to the GatewayServer, again using TCP communication.

Thus, the LeaderServer functions as a routing and load-distribution component within the distributed architecture.

Worker Servers

WorkerServer instances are responsible for executing the core business logic of the system. Each WorkerServer contains a set of specialized microservice objects (see below) that implement different categories of operations.

When a WorkerServer receives a request from the LeaderServer, it determines which microservice is responsible for handling the request and forwards the request accordingly. Once the microservice completes its task, the WorkerServer sends the result back to the LeaderServer via TCP.

This design allows each WorkerServer to coordinate multiple independent service components while maintaining a consistent interface for request processing.

Shown below in [Figure 3](#) is a more detailed view of the individual microservices that each WorkerService coordinates and interacts with:

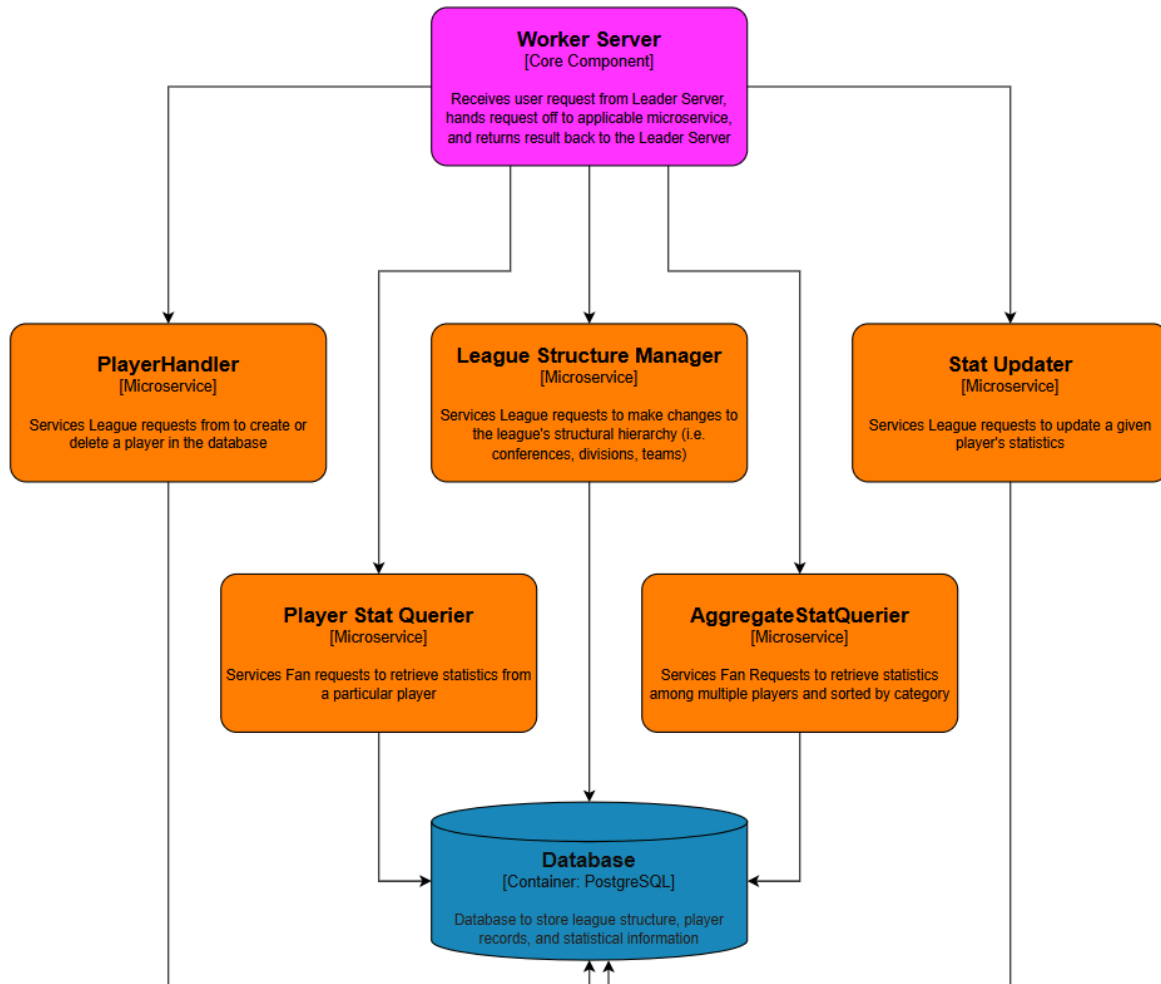


Figure 3

(Note: this figure depicts order of operations among system components from WorkerServer to database when a request is made. When the request is fulfilled, the result is propagated from database to WorkerServer in the opposite direction i.e. arrows flipped)

Microservices

Each WorkerServer contains several microservice objects that perform specific operations on the system's data. These services interact directly with the underlying PostgreSQL database.

The PlayerHandler microservice is responsible for managing player records. It supports operations that create new players and delete existing players from the database.

The LeagueStructureManager microservice manages the hierarchical organization of the league. It supports operations such as moving players between teams, moving teams between divisions, and moving divisions between conferences. It also supports creating and deleting teams, divisions, and conferences within the database.

The StatUpdater microservice manages updates to player statistics. For example, for MLB it would provide methods for incrementing various baseball statistics, such as at-bats, hits, home runs, runs batted in, stolen bases, walks, and other related metrics. It also provides methods for setting the value of these statistics directly.

The PlayerStatQuerier microservice provides read operations for retrieving statistics associated with individual players. It can return either a specific statistic or the complete set of statistics for a given player.

The AggregateStatQuerier microservice supports more complex queries that aggregate data across multiple players. These queries allow users to retrieve ranked results for a particular statistical category within a specified level of the league hierarchy, such as conference, division, or team, and can be limited to a specified number of top players within a given year.

Database Layer

All microservices interact with a shared (across all WorkerServers) PostgreSQL database cluster managed by Patroni, which provides automatic failover and high availability. The cluster maintains a primary node and one or more replicas kept in sync with the primary; if the primary fails, Patroni automatically promotes the most up-to-date replica to take its place, ensuring the database remains available. This database stores all persistent data for the system, including league structure, player records, and statistical information. Write operations performed by administrative users modify the database through the appropriate microservice, while read operations retrieve data for fan queries.

Request and Response Flow

The overall flow of a request through the system follows a consistent pattern.

A user first initiates an operation through either the LeagueInterface or the FanInterface. The application interface sends the request as an HTTP message to the GatewayServer. The gateway forwards the request to the LeaderServer via TCP. The LeaderServer selects the next WorkerServer in its round-robin rotation and routes the request to it via TCP. The WorkerServer then invokes the relevant microservice, which performs the required database operation.

Once the microservice finishes executing the operation, the result is returned to the WorkerServer. The WorkerServer sends the result to the LeaderServer via TCP, which forwards it to the GatewayServer via TCP. Finally, the GatewayServer returns the result to the original client interface as an HTTP response, where the user can view it.

Thus, each request follows a complete round-trip path through the system.

Below in [Figure 4](#) is a complete workflow of the entire system architecture:

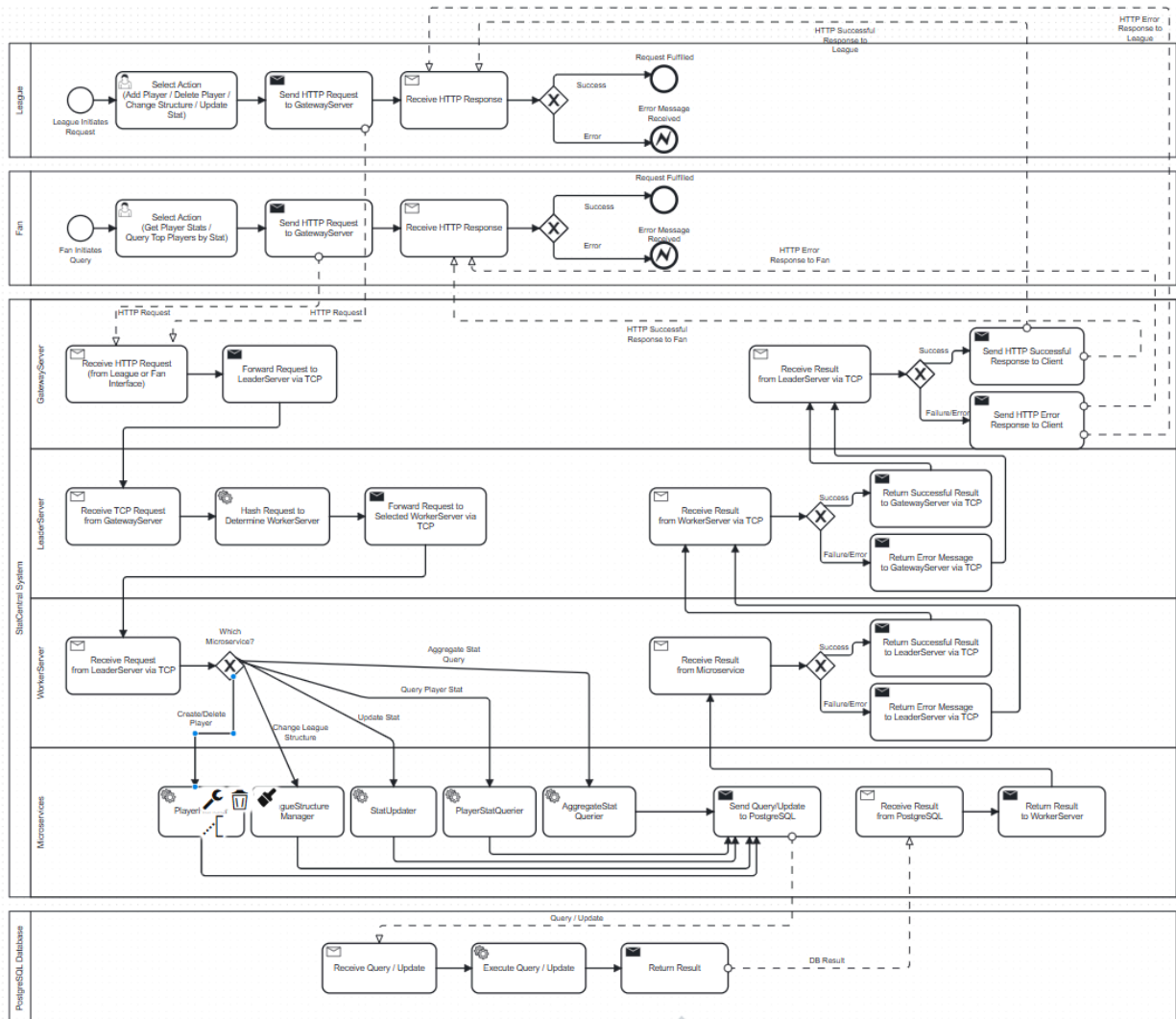


Figure 4

Error Handling

Failures may occur at various points in the request processing pipeline. These failures can include invalid operations, database errors, or system-level communication issues. Whenever an error occurs during processing, the component where the failure occurs generates an error message. This error is propagated back up the same chain of components that handled the request, ensuring that the originating client receives a response indicating the failure.

Because each TCP request expects a corresponding TCP response and each HTTP request expects a corresponding HTTP response, the system guarantees that the client always receives either a successful result or an error message.

Success Scenarios:

- League creates the team “New York Yankees”
- League creates Aaron Judge’s player record ([Figure 5](#))
- League submits an update request on Aaron Judge’s ‘hit’ count
- Fan requests statistics for a particular player
- Fan makes an aggregate statistics query for “top 10 home run hitters in the AL in 2024”

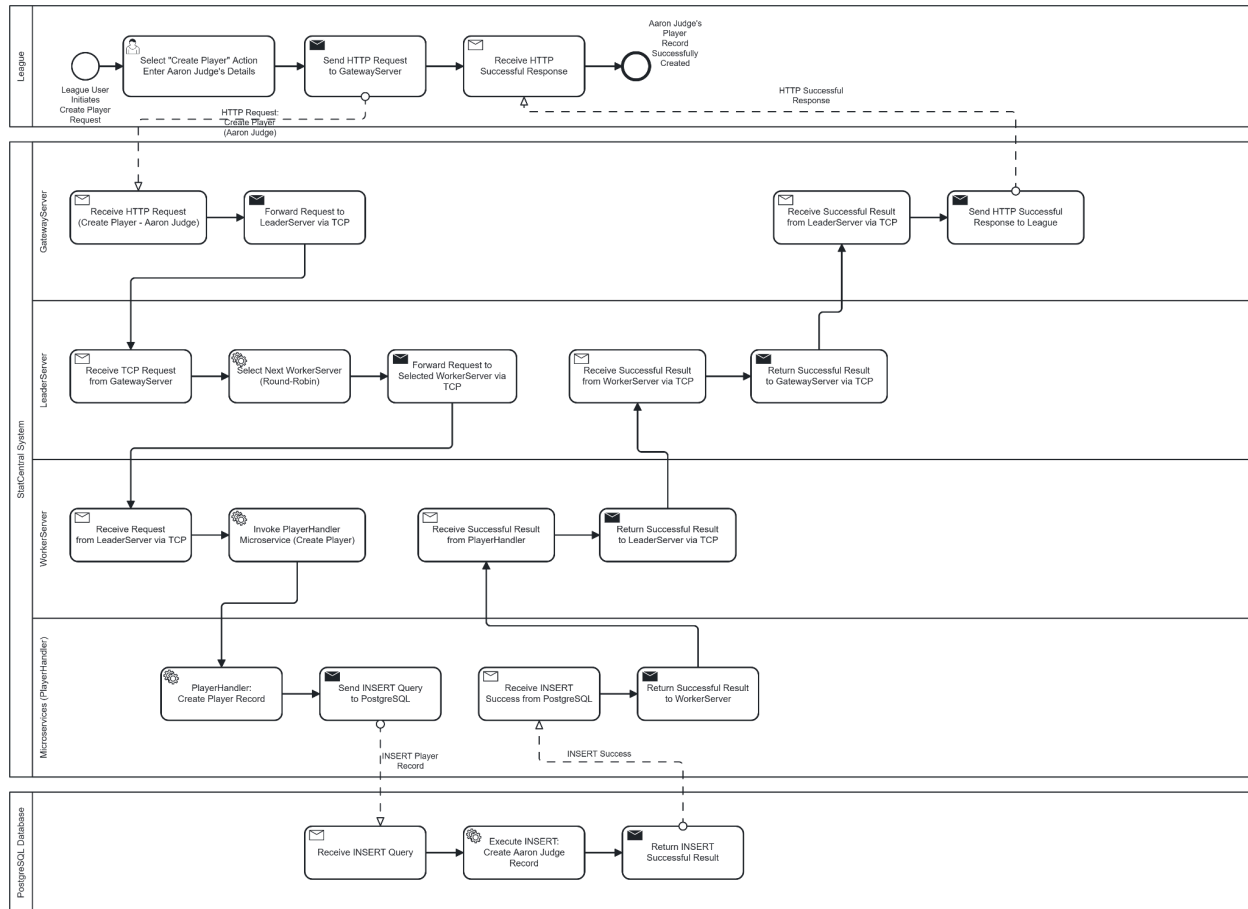


Figure 5

Failure Scenarios:

- User input failure scenarios:
 - Fan requests statistics for a player that is not in the database
 - Fan makes an aggregate statistics query based on a statistical category that doesn't exist in (is not defined by) the system ([Figure 6](#))
 - League submits an update request that includes values that violate logical constraints on player statistics (e.g. negative value for a statistic such as “homeRuns”)
 - League attempts to create a player record of a player that already exists in the database
 - League attempts to delete a team that doesn't exist in the database

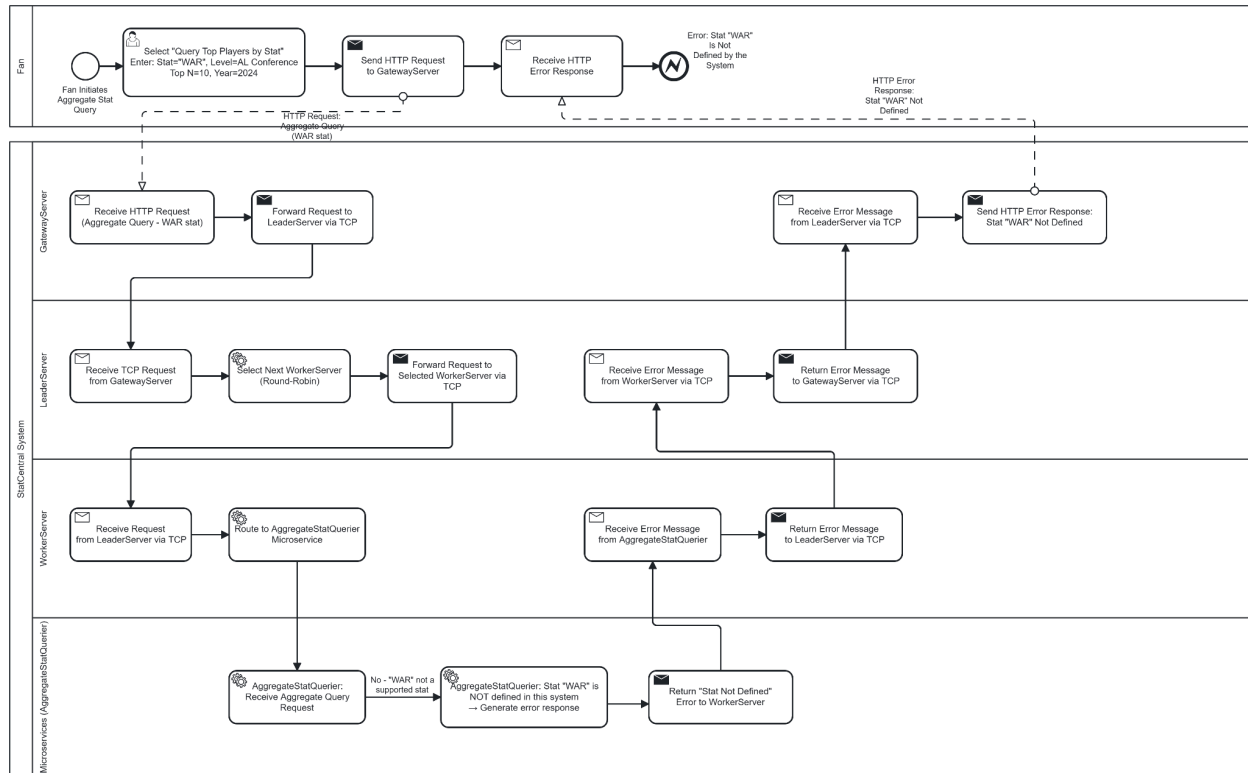


Figure 6

- System failure scenarios:
 - The GatewayServer crashes, and the LeagueInterface and FanInterface are unable to submit requests to the system. Upon detecting GatewayServer failure, the interface starts up a new GatewayServer. The new GatewayServer must first communicate with the backend cluster to identify the current leader before it can forward any requests. Once the new GatewayServer has established which node is the current leader, the user interface (LeagueInterface or FanInterface) resends all failed requests to the new GatewayServer for processing.
 - The LeaderServer is detected as failed by the other nodes in the cluster (the GatewayServer and WorkerServers), and an election is initiated to elect one of the live WorkerServers as the new LeaderServer. The GatewayServer, upon learning which server was elected as the new LeaderServer, resumes forwarding requests to that server. Any requests that were in-flight at the time of the crash are re-issued by the GatewayServer once the new leader is ready.
 - A WorkerServer is detected as failed by the LeaderServer while processing requests. For all requests that are incomplete at the time of the failure, the requests will be resent to another live WorkerServer, which will successfully fulfill the request ([Figure 7](#)).
 - Within a WorkerServer, individual microservices may experience failures. In this scenario, the requests will be detected as failed by the WorkerServer, and an error message will be returned to the LeaderServer. The LeaderServer will then reroute the failed requests to another WorkerServer whose microservices can complete the requests.

- The primary PostgreSQL node fails. Patroni detects the failure and automatically promotes the most up-to-date replica to primary. During this brief failover window, the microservice receives a transient connection error. It applies a short retry with backoff, and by the time the retry is issued, the new Patroni-promoted primary is ready to receive it. The query executes successfully on the new primary, and the result propagates back to the client normally ([Figure 8](#)).

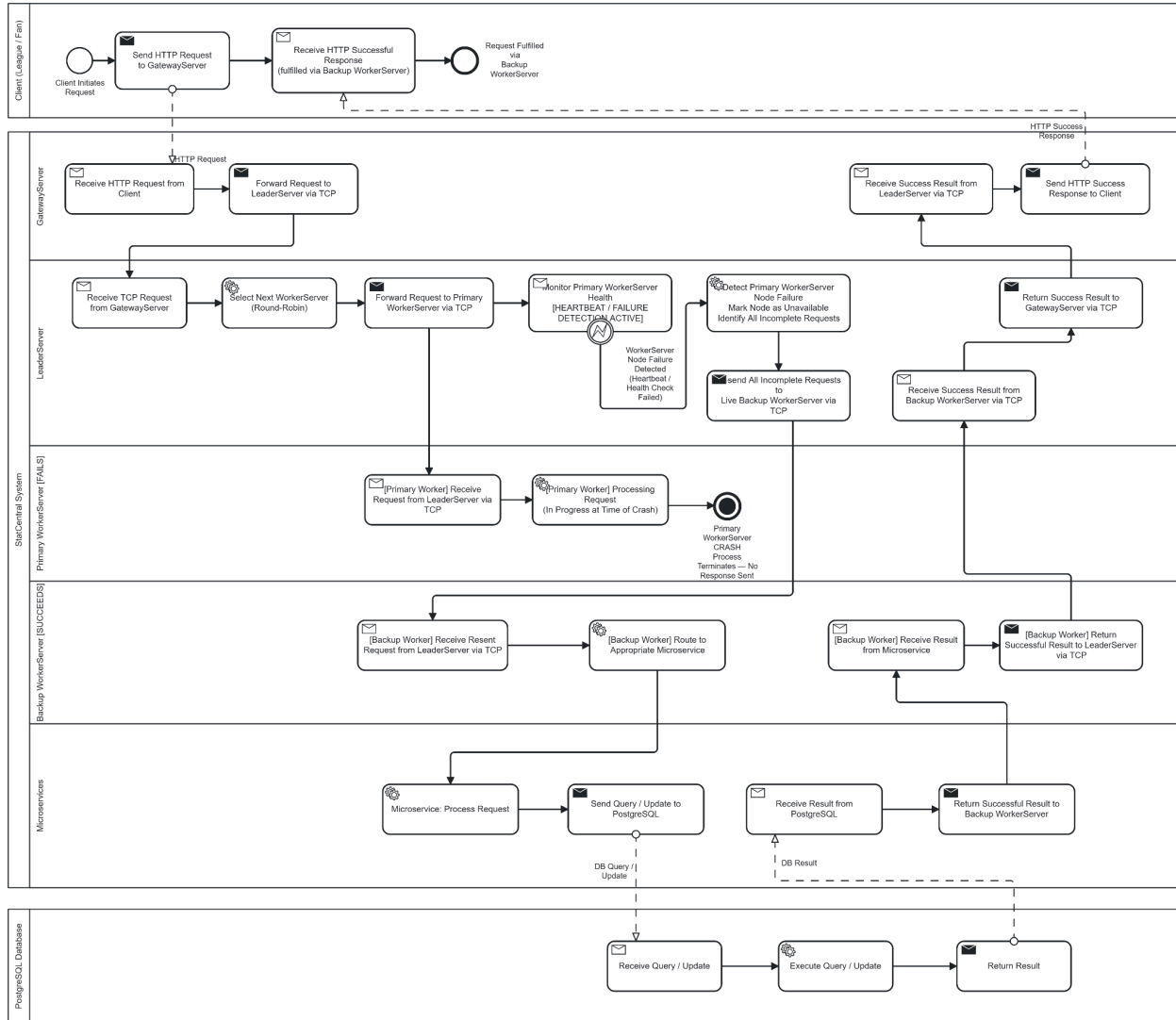


Figure 7

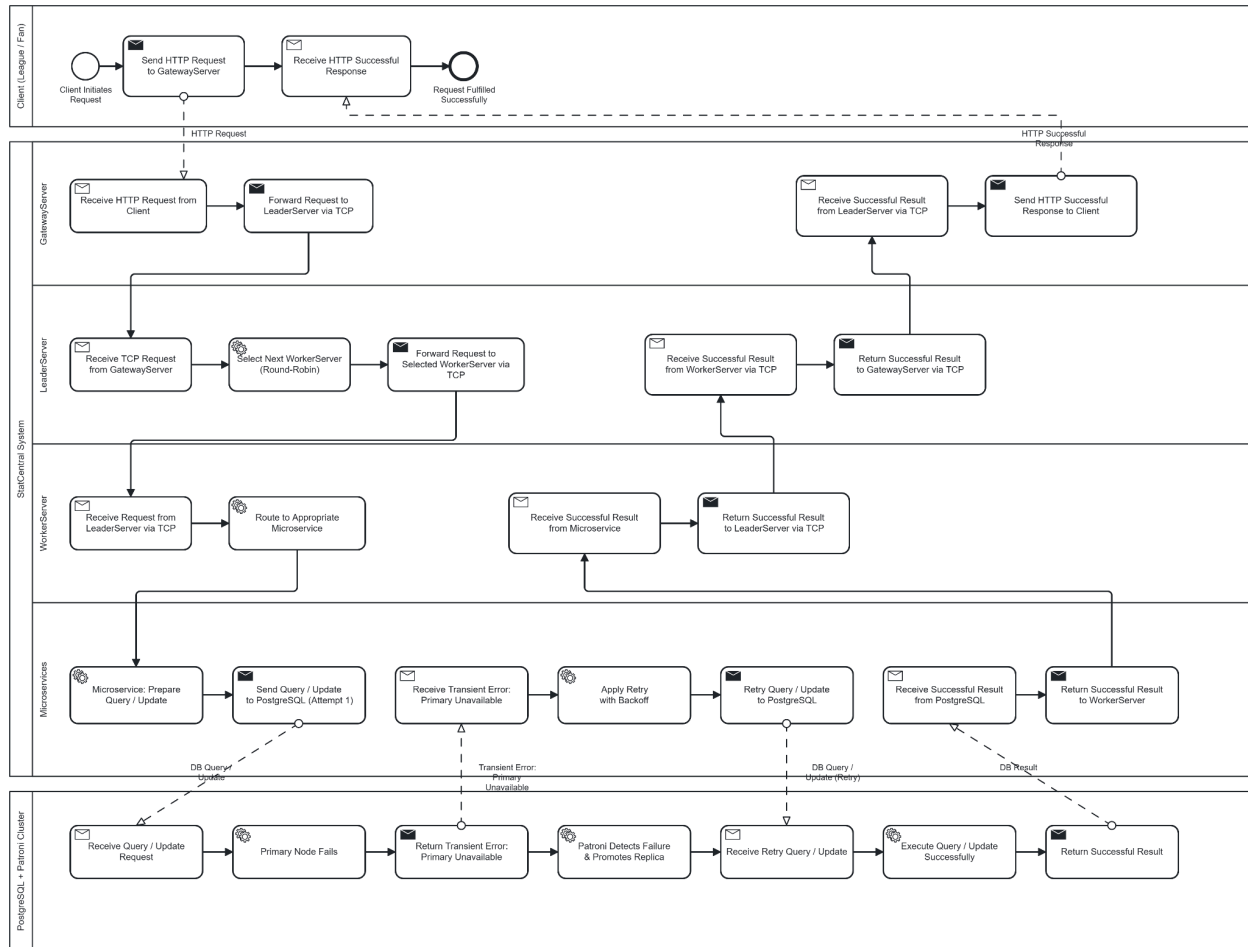


Figure 8

Section III: System Design – Deeper Level

Overview

This section provides a detailed walkthrough of the logical steps each system component must execute in order to process each type of request. For every request type, the following are described: the content of the request, the desired successful response, the sequential steps along the happy path, and the possible problems that may arise at each step along with the strategies the system uses to prevent or recover from those problems.

Requests originate from one of two client interfaces — the `LeagueInterface` or the `FanInterface` — and travel through a consistent pipeline: `User Interface` → `GatewayServer` → `LeaderServer` → `WorkerServer` → `Microservice` → `PostgreSQL Database` (Cluster managed by `Patroni`). Results propagate back through the same chain in reverse.

The request types addressed in this section are organized into three groups:

- League Structure Management Requests (via `LeagueInterface`): `createConference`, `createDivision`, `createTeam`, `deleteConference`, `deleteDivision`, `deleteTeam`, `movePlayerToTeam`, `moveTeamToDivision`, `moveDivisionToConference`
- Player Management & Stat Update Requests (via `LeagueInterface`): `createPlayer`, `deletePlayer`, `updatePlayerStat`
- Fan Query Requests (via `FanInterface`): `getPlayerStats`, `queryTopPlayersByStat`

Before detailing the portions of the happy paths that are specific to each of these groups, as well as the potential problems that may arise for each one, I will detail the steps of the pipeline which are common among all three groups (sub-section A). Following this, I will dive into the happy paths and potential problems specific to each group (sub-sections B-D). In the final section (section E), I will provide state diagrams to provide a visual for the operations and states of the major components of the system.

A. General Steps and Failure Scenarios

Most of the steps in the system workflow are common steps, so I have included them in this sub-section as a reference for later sub-sections. What makes each operation unique is in the actions performed by the various microservices, as explained in greater detail in later sections' Step 5.

Happy Path – General Steps

Step 1: User Interface Constructs and Sends HTTP Request

- The league administrator invokes the appropriate method on the applicable user interface (`LeagueInterface` or `FanInterface`)
- The user interface serializes the method name and parameters into an HTTP request body.
- The request is sent asynchronously via HTTP to the `GatewayServer`.

Step 2: GatewayServer Receives, Validates, and Forwards the Request

- The GatewayServer receives the HTTP request from the user interface.
- The GatewayServer validates the structure of the request, wraps it into a Request object, and asynchronously forwards it to the LeaderServer via TCP.
- The GatewayServer maintains a reference to the originating HTTP response object so it can respond to the client once the result is returned.

Step 3: LeaderServer Receives and Routes the Request

- The LeaderServer receives the Request object from the GatewayServer via TCP.
- It selects the next WorkerServer using a round-robin scheduling strategy, advancing an internal counter with each request.
- The selected WorkerServer index is computed from the current position in the rotation (e.g., using modular arithmetic: $\text{requestCount} \% \text{numberOfWorkers}$).
- The LeaderServer forwards the request to the selected WorkerServer via TCP.

Step 4: WorkerServer Receives the Request and Delegates to a Microservice

- The WorkerServer receives the request from the LeaderServer via TCP.
- It inspects the operation type identifier in the request body to determine which microservice should handle it.
- The WorkerServer delegates the request to the appropriate microservice by calling the applicable method.

Step 5: Microservice Executes Database Query

- This operation is dependent on the microservice and query to that microservice. See sections B-D for greater detail.

Step 6: Result Propagates Back to Client

- The WorkerServer receives the result from the applicable microservice and sends it back to the LeaderServer via TCP.
- The LeaderServer receives the result and forwards it to the GatewayServer via TCP.
- The GatewayServer receives the result and sends it as an HTTP response back to the originating user interface.
- The user interface delivers the result to the caller.

Possible Problems and System Responses

Step 1 Problem – User Interface

- **GatewayServer becomes unavailable (detected via gossip):** The LeagueInterface and FanInterface participate in a gossip-based failure detection protocol, continuously monitoring for heartbeat messages from the GatewayServer. If the user interface does not receive a gossip message from the GatewayServer within a configured time interval, it marks the GatewayServer as dead. To handle in-flight requests that were sent to the now-dead GatewayServer but never received a response, both interfaces maintain a local cache of all in-flight requests. Upon detecting GatewayServer failure, the user

interface starts up a new GatewayServer. The new GatewayServer must first communicate with the backend cluster to identify the current leader before it can forward any requests. Once the new GatewayServer has established which node is the current leader, the user interface resends all cached in-flight requests to the new GatewayServer for processing.

Step 2 Problem – GatewayServer

- **LeaderServer crashes:** All nodes in the backend cluster (the LeaderServer, WorkerServers, and the GatewayServer) continuously exchange gossip messages with each other. Each node tracks the last time it received a gossip message from every other node. If a node does not receive a gossip message from another node within a configured time interval, it marks that node as dead. When the LeaderServer is marked dead by its peers in this way, a leader election is automatically triggered. All surviving peer nodes in the cluster participate in a vote to elect a new leader. Once a new leader is elected, it takes over responsibility for routing requests. The GatewayServer, upon learning of the new leader's identity through the gossip protocol, resumes forwarding requests to the newly elected leader. Any requests that were in-flight at the time of the crash are re-issued by the GatewayServer once the new leader is ready.

Step 3 Problem – LeaderServer

- **Selected WorkerServer crashes:** The LeaderServer monitors pending requests sent to each WorkerServer. If the LeaderServer detects a WorkerServer as failed (via the gossip detection protocol explained earlier), it reroutes all of its incomplete requests to another live WorkerServer. Since all WorkerServers contain the same microservices and share the same PostgreSQL database, the backup can complete the operation.

Step 4 Problems – WorkerServer

- **Microservice is unresponsive:** The WorkerServer returns an error message to the LeaderServer. The LeaderServer then reroutes the request to another WorkerServer whose microservice can complete it.
- **Unrecognized operation type reaching the WorkerServer:** If the request body contains a malformed or unrecognized operation type identifier, the WorkerServer cannot determine which microservice to delegate to and returns an error message to the LeaderServer, which propagates it back to the client.

Step 5 Problems – Microservice

- **Database node failure (handled by PostgreSQL + Patroni replication):** The database layer is implemented as a replicated PostgreSQL cluster managed by Patroni. Patroni continuously monitors the health of all PostgreSQL nodes and automatically promotes a replica to primary if the current primary fails. Because the cluster maintains synchronized replicas, a single node failure does not result in database unavailability. Microservices connecting to the database connect through a Patroni-managed endpoint that always points to the current primary. In the event of a failover, the endpoint is updated by Patroni and connections are re-established transparently. Only during the brief failover window (between the primary's failure and the promotion of a replica) might a microservice receive a transient connection error. In that case, the microservice should apply a short retry with backoff before propagating a failure message to the WorkerServer.

B. League Structure Management Requests

The requests in this group all originate from the LeagueInterface, travel the same component pipeline, and are ultimately handled by the LeagueStructureManager microservice. Because they share an identical pipeline structure, a unified pipeline walkthrough is presented below, followed by request-specific notes where the business logic or failure modes differ.

Request Types and Their Content

Create a new Conference

- Content of request: operation type identifier ("createConference"), conference name
- Desired response: success message confirming the conference was created, or an error message if a conference with that name already exists

Create a new Division

- Content of request: operation type identifier ("createDivision"), division name, parent conference name
- Desired response: success message confirming the division was created, or an error message if the conference does not exist or a division with that name already exists within it

Create a new Team

- Content of request: operation type identifier ("createTeam"), team name, parent division name
- Desired response: success message confirming the team was created, or an error message if the division does not exist or a team with that name already exists within it

Delete an existing Conference

- Content of request: operation type identifier ("deleteConference"), conference name
- Desired response: success message confirming the conference and all of its cascaded contents (divisions, teams, players) were deleted, or an error message if the conference does not exist

Delete an existing Division

- Content of request: operation type identifier ("deleteDivision"), division name
- Desired response: success message confirming the division and all of its cascaded contents (teams, players) were deleted, or an error message if the division does not exist

Delete an existing Team

- Content of request: operation type identifier ("deleteTeam"), team name
- Desired response: success message confirming the team and all of its players were deleted, or an error message if the team does not exist

Move a specific Player to a specific Team

- Content of request: operation type identifier ("movePlayerToTeam"), player ID, target team name
- Desired response: success message confirming the player was moved, or an error message if the player or target team does not exist

Move a specific Team to a specific Division

- Content of request: operation type identifier ("moveTeamToDivision"), team name, target division name
- Desired response: success message confirming the team was moved, or an error message if the team or target division does not exist

Move a specific Division to a specific Conference

- Content of request: operation type identifier ("moveDivisionToConference"), division name, target conference name
- Desired response: success message confirming the division was moved, or an error message if the division or target conference does not exist

Happy Path – League Structure Management Request

The following steps describe the happy path for any league structure management request from submission through database execution.

Steps 1-3: User Interface -> GatewayServer -> LeaderServer -> WorkerServer (Sub-Section A)

- The appropriate method on the LeagueInterface for this operation is called in Step 1 (e.g., createTeam("Yankees", "AL East")).

Step 4: WorkerServer Receives the Request and Delegates to LeagueStructureManager

- The WorkerServer delegates to its LeagueStructureManager microservice by calling LeagueStructureManager.createTeam(teamName, divisionName).

Step 5: LeagueStructureManager Executes Database Operation

- The LeagueStructureManager prepares and executes the appropriate SQL query against the PostgreSQL database.
- For create operations: an INSERT statement is issued. For delete operations: a DELETE statement with cascading constraints is issued (see comments in LeagueStructureManager class). For move operations: an UPDATE statement is issued to change the parent reference of the entity being moved.
- The database returns a successful result or an error (e.g., duplicate key violation for a create, or no rows affected for a delete of a nonexistent entity).
- The LeagueStructureManager returns the result (success or error message) to the WorkerServer.

Step 6: Result Propagates Back to Client (Sub-Section A)

- The applicable microservice that the result propagates back from is the LeagueStructureManager

Possible Problems and System Responses

Step 5 Problems – LeagueStructureManager and Database

- **Entity already exists (uniqueness violation on create operations):** The database returns a constraint violation error. The LeagueStructureManager catches this, formats an appropriate error message (e.g., "Conference 'AL' already exists"), and returns it to the WorkerServer for propagation to the client.
- **Entity does not exist (delete or move on a nonexistent entity):** The DELETE or UPDATE statement affects zero rows. The LeagueStructureManager detects this and returns an appropriate error message.
- **Concurrent conflicting write operations (e.g., two workers attempting to create the same conference simultaneously):** Since all WorkerServers share the same PostgreSQL database, and the database enforces uniqueness constraints, one of the two concurrent writes will succeed and the other will receive a constraint violation error. The losing request is returned to the client as an error. No additional distributed locking is required for these operations given the database's transactional guarantees.

C. Player Management and Stat Update Requests

This group covers three request types originating from the LeagueInterface: createPlayer (handled by PlayerHandler), deletePlayer (handled by PlayerHandler), and updatePlayerStat (handled by StatUpdater). The pipeline is identical to the one described in Section A, but the microservice responsible for execution changes, and the business logic and failure modes differ accordingly

Request Types and Their Content

Create a new Player

- Content of request: operation type identifier ("createPlayer"), PlayerData object containing player name, team name, and all statistical fields
- Desired response: success message confirming the player record was created, or an error message if the specified team does not exist in the league structure or if a player with that name already exists

Delete an existing Player

- Content of request: operation type identifier ("deletePlayer"), player ID (player name)
- Desired response: success message confirming the player was deleted, or an error message if the player does not exist

Update a specific Stat category for a specific Player

- Content of request: operation type identifier ("updatePlayerStat"), player ID, stat name (e.g., "homeRun"), integer value
- Desired response: success message confirming the stat was updated, or an error message if the player does not exist or the stat name is not defined in the system

Happy Path – Create a new Player

Steps 1-3: User Interface -> GatewayServer -> LeaderServer -> WorkerServer (Sub-Section A)

- The appropriate method on the LeagueInterface for this operation is createPlayer(playerData)

Step 4: WorkerServer Receives the Request and Delegates to PlayerHandler

- The WorkerServer delegates to its PlayerHandler microservice by calling PlayerHandler.createPlayer(playerData)

Step 5: PlayerHandler Validates League Structure and Inserts Player

- PlayerHandler first queries the database to verify that the team named in the PlayerData object exists in the league structure.
- If the team exists, PlayerHandler issues an INSERT statement to create the player record with all provided statistical field values.
- The database returns a successful result or a constraint violation if the player already exists.
- PlayerHandler returns a result message to the WorkerServer.

Step 6: Result Propagates Back to Client (Sub-Section A)

- The applicable microservice that the result propagates back from is the PlayerHandler

Happy Path – Delete an existing Player

Steps 1-3: User Interface -> GatewayServer -> LeaderServer -> WorkerServer (Sub-Section A)

- The appropriate method on the LeagueInterface for this operation is deletePlayer(playerId)

Step 4: WorkerServer Receives the Request and Delegates to PlayerHandler

- The WorkerServer delegates to its PlayerHandler microservice by calling PlayerHandler.deletePlayer(playerName)

Step 5: PlayerHandler Deletes Player Record

- PlayerHandler issues a DELETE statement targeting the player record with the given player name.
- If no record is found, an error message is returned. Otherwise, a success message is returned.

Step 6: Result Propagates Back to Client (Sub-Section A)

- The applicable microservice that the result propagates back from is the PlayerHandler

Happy Path – Update a specific Stat category for a specific Player

Steps 1-3: User Interface -> GatewayServer -> LeaderServer -> WorkerServer (Sub-Section A)

- The appropriate method on the LeagueInterface for this operation is updatePlayerStat(playerId, statName, value)

Step 4: WorkerServer Receives the Request and Delegates to StatUpdater

- The WorkerServer receives the request from the LeaderServer, identifies it as an "updatePlayerStat" request, and uses the leagueId field on the Request object to look up the correct StatUpdater instance from its internal registry (e.g., leagueId "MLB" resolves to the MLBStatUpdater instance, "NFL" resolves to the NFLStatUpdater instance, etc.).
- The WorkerServer then calls the resolved StatUpdater's incrementStat(playerId, statName) method for increment-mode updates, or setStat(playerId, statName, value) for direct-value-set updates. Both methods are defined on the abstract StatUpdater base class, which validates the stat name against the sport-specific set of supported stats before delegating to the concrete implementation.

Step 5: StatUpdater Executes Database Update

- StatUpdater issues an UPDATE statement to set or increment the appropriate statistical field in the player's database record.
- If the player does not exist, the UPDATE affects zero rows, and an error message is returned. Otherwise, a success message is returned.

Step 6: Result Propagates Back to Client (Sub-Section A)

- The applicable microservice that the result propagates back from is the StatUpdater

Possible Problems and System Responses

Note: Problems that may occur during Steps 1 through 3 (LeagueInterface, GatewayServer, and LeaderServer) are identical for createPlayer, deletePlayer, and updatePlayerStat, and are the same as those described for Section A. Only the microservice-layer problems (Steps 4 and 5) differ per request type and are documented below.

Create a new Player – Step 5 Problems

- **Team does not exist in the league structure:** PlayerHandler queries the database for the team before attempting the insert. If no matching team record is found, it returns an error message without issuing the INSERT. This two-step read-then-write introduces a TOCTOU (time-of-check time-of-use) race condition: if the team is deleted between the read check and the INSERT, the INSERT will fail with a foreign key constraint violation. PlayerHandler will handle this exception and return an appropriate error message.
- **Player already exists (duplicate name):** The database returns a uniqueness constraint violation. PlayerHandler catches this and returns an error message to the caller.
- **PlayerData contains null or empty required fields not caught at the GatewayServer:** PlayerHandler performs its own validation of the PlayerData before issuing the INSERT. If required fields are missing, it returns an error message.

Delete an existing Player – Step 4 Problems

- **Player ID is null or empty:** If the player ID extracted from the request body is null or empty at the WorkerServer level, the WorkerServer returns an error message to the LeaderServer which is then propagated back to the client.

Delete an existing Player – Step 5 Problems

- **Player does not exist:** The DELETE returns zero rows affected. PlayerHandler detects this condition and returns an error message.
- **Concurrent deleteTeam and deletePlayer for the same player's team:** If a team is deleted (cascading to its players) at the same time as a separate deletePlayer request is in flight for one of those players, the PostgreSQL database's transaction isolation ensures only one operation wins. The concurrent deletePlayer request may return either a success (if it completes before the cascade) or a "player not found" error (if the cascade completes first). Both outcomes are acceptable.

Update a specific Stat category for a specific Player – Step 4 Problems

- **Unrecognized leagueId (no StatUpdater registered for this league):** If the leagueId on the Request does not match any entry in the WorkerServer's StatUpdater registry, the WorkerServer cannot resolve a StatUpdater and returns an error message to the LeaderServer (e.g., "No StatUpdater registered for league 'XYZ'"), which propagates it back to the client. This would occur if the leagueId in the request was corrupted or for a league which is not in the system at the time of the request.
- **Unrecognized stat name:** The abstract StatUpdater base class validates the stat name against the set of supported stats registered by the sport-specific subclass (e.g., MLBStatUpdater) before delegating to the database update logic. If the stat name is not in the supported set, the base class returns an error message to the WorkerServer indicating the stat category is not defined in the system, and this error message is propagated back up to the client.
- **Negative value passed for a stat that cannot be negative:** The abstract StatUpdater base class validates that the value is non-negative before delegating to the sport-specific implementation. If the value violates this constraint, the base class returns an error message without modifying the database. This error message is propagated back up to the client via the WorkerServer.

Update a specific Stat category for a specific Player – Step 5 Problems

- **Concurrent stat updates for the same player (e.g., two officials updating homeRun simultaneously):** Because the UPDATE operates on a single row and PostgreSQL provides row-level locking, concurrent updates to the same player's stat are serialized at the database layer. Both updates will eventually succeed in sequence. For increment operations, this means the final value reflects both increments. No additional application-level locking is required.
- **Player does not exist:** The UPDATE affects zero rows. StatUpdater detects this and returns an error message. This error message is propagated back up to the client via the WorkerServer.

D. Fan Query Requests

This group covers two request types originating from the FanInterface: getPlayerStats (handled by PlayerStatQuerier) and queryTopPlayersByStat (handled by AggregateStatQuerier). These are read-only operations that do not modify the database. The pipeline is the same as described in prior sections, but because these are reads rather than writes, the concurrency concerns are different.

Request Types and Their Content

Get all of a specific Player's Stats

- Content of request: operation type identifier ("getPlayerStats"), player ID (player name)
- Desired response: a formatted string containing all statistical values for the given player, or an error message if the player does not exist

Query the top Players by a specific Stat category

- Content of request: operation type identifier ("queryTopPlayersByStat"), stat name, league level ("conference", "division", or "team"), league level value (e.g., "AL", "East", "Yankees"), topN (integer), year (integer)
- Desired response: a ranked list of the top N players and their stat values for the given filters, or an error message if the stat name, league level value, or year is not defined in the system

Happy Path – Get all of a specific Player's Stats

Steps 1-3: User Interface -> GatewayServer -> LeaderServer -> WorkerServer (Sub-Section A)

- The appropriate method on the FanInterface for this operation is getPlayerStats(playerId)

Step 4: WorkerServer Receives the Request and Delegates to PlayerStatQuerier

- The WorkerServer delegates to its PlayerStatQuerier microservice by calling PlayerStatQuerier.getAllPlayerStats(playerName)

Step 5: PlayerStatQuerier Queries Database

- The PlayerStatQuerier issues a SELECT statement to retrieve all statistical fields for the player with the given name.
- The result rows are formatted into a result string and returned to the WorkerServer.

Step 6: Result Propagates Back to Client (Sub-Section A)

- The applicable microservice that the result propagates back from is the PlayerStatQuerier

Happy Path – Query the top Players by a specific Stat category

Steps 1-3: User Interface -> GatewayServer -> LeaderServer -> WorkerServer (Sub-Section A)

- The appropriate method on the FanInterface for this operation is `queryTopPlayersByStat(statName, leagueLevel, leagueLevelValue, topN, year)`

Step 4: WorkerServer Receives the Request and Delegates to AggregateStatQuerier

- The WorkerServer delegates to its AggregateStatQuerier microservice by calling `AggregateStatQuerier.queryTopPlayersByStat(statName, leagueLevel, leagueLevelValue, topN, year)`

Step 5: AggregateStatQuerier Executes Complex Query

- The AggregateStatQuerier constructs an SQL query that JOINS the player table with the league structure tables (conferences, divisions, teams) to filter players belonging to the specified league level value.
- The query selects the specified stat column for the specified year, orders the results in descending order, and limits the output to topN rows.
- The database returns the ranked result set.
- The AggregateStatQuerier formats the results into a ranked list string and returns it to the WorkerServer.

Step 6: Result Propagates Back to Client (Sub-Section A)

- The applicable microservice that the result propagates back from is the AggregateStatQuerier

Possible Problems and System Responses

Get all of a specific Player's Stats – Step 5 Problems

- **Player does not exist:** The SELECT returns zero rows. The PlayerStatQuerier detects this and returns an error message ("Player not found") to the WorkerServer.
- **Read-write concurrency (stat being updated by the league while a fan queries it):** PostgreSQL's default read committed isolation level ensures that the fan query will always see a consistent committed state of the data. It will not read partially updated rows or dirty data. A fan may see a stat value that is one increment behind a concurrent update — this is an acceptable and expected behavior for a statistics query system.

Query the top Players by a specific Stat category – Step 5 Problems

- **stat name is not defined in the system:** The AggregateStatQuerier validates that the `statName` corresponds to a known database column before executing the query. If not recognized, it returns an error message ("Stat category not defined") without executing the query. This also prevents SQL injection via the stat name parameter.
- **leagueLevelValue does not exist (e.g., conference 'Hufflepuff' not in the database):** The JOIN query returns zero rows. The AggregateStatQuerier detects an empty result set and returns an error message indicating the specified league level value does not exist.

- **No data exists for the specified year:** The query filters by year and returns zero rows. The AggregateStatQuerier returns an appropriate error message indicating no statistical data is available for that year.
- **Query returns fewer than topN results (e.g., only 3 players in a team when topN=10):** The AggregateStatQuerier returns however many results are available. This is not an error condition; the system returns a partial ranked list with a count fewer than topN.
- **Concurrent structural change while aggregate query is executing (e.g., a team being moved to a different division mid-query):** PostgreSQL's transaction isolation ensures the query sees a consistent snapshot of the data at the time it begins. The result may not reflect the very latest structural change if that change commits after the query begins, but the query will not return an inconsistent or partially-updated view.

E. State Diagrams

In this final section I will present state diagrams for each major component of the system. Each of these diagrams gives a picture of the success and failure scenarios described in the previous sections.

Diagram I – User Interface

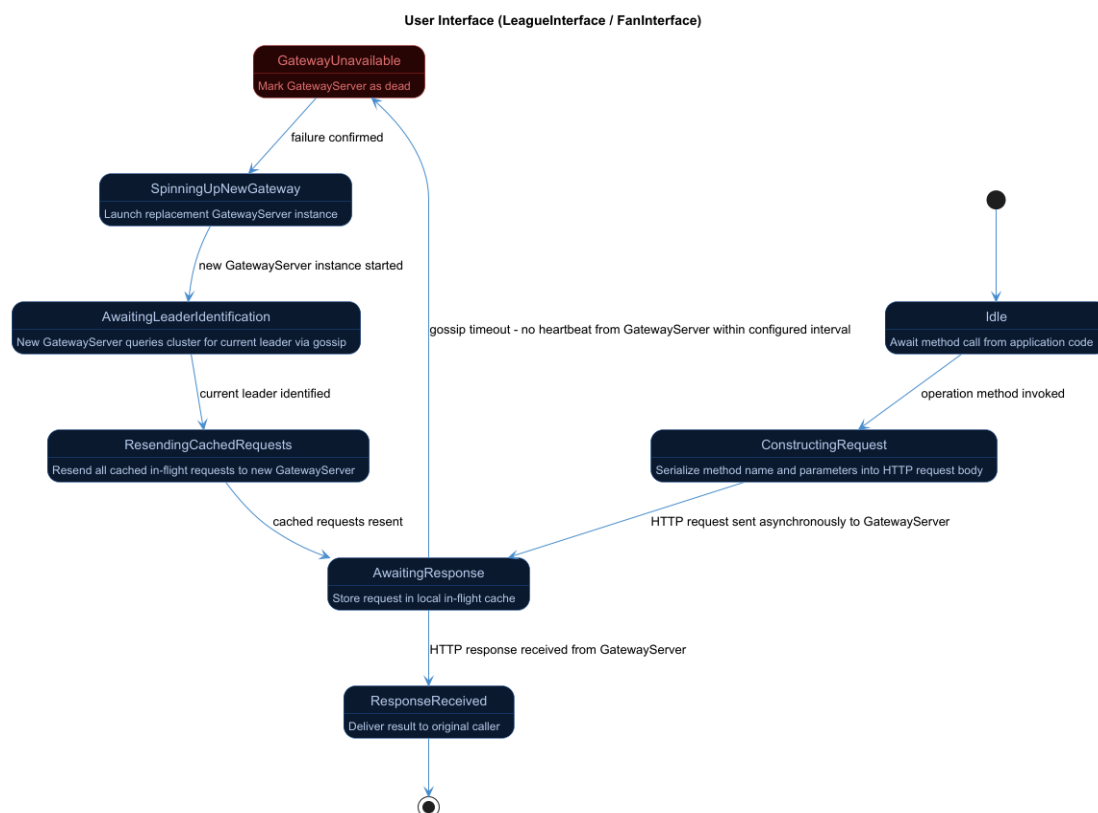


Diagram II – GatewayServer

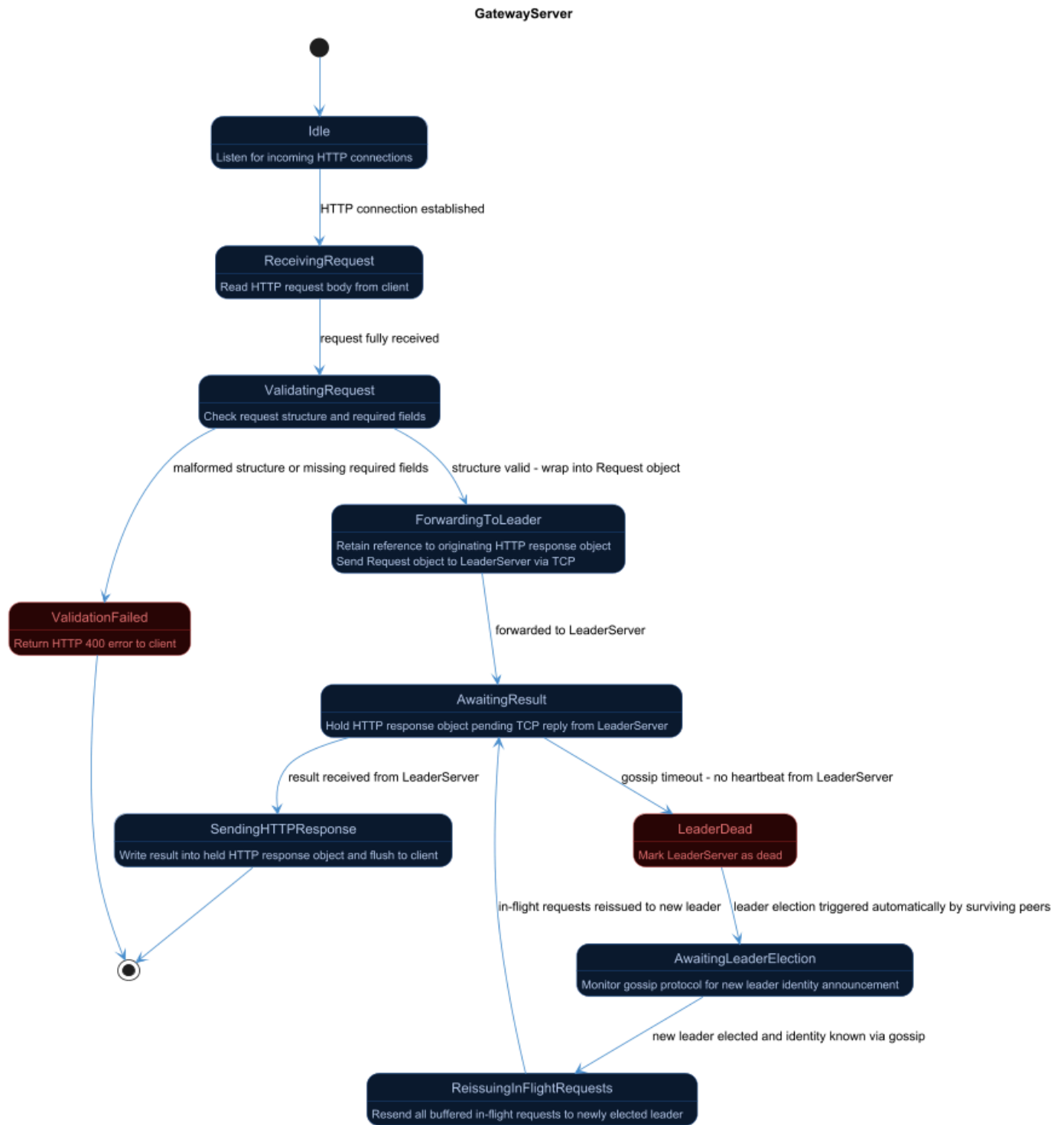


Diagram III – LeaderServer

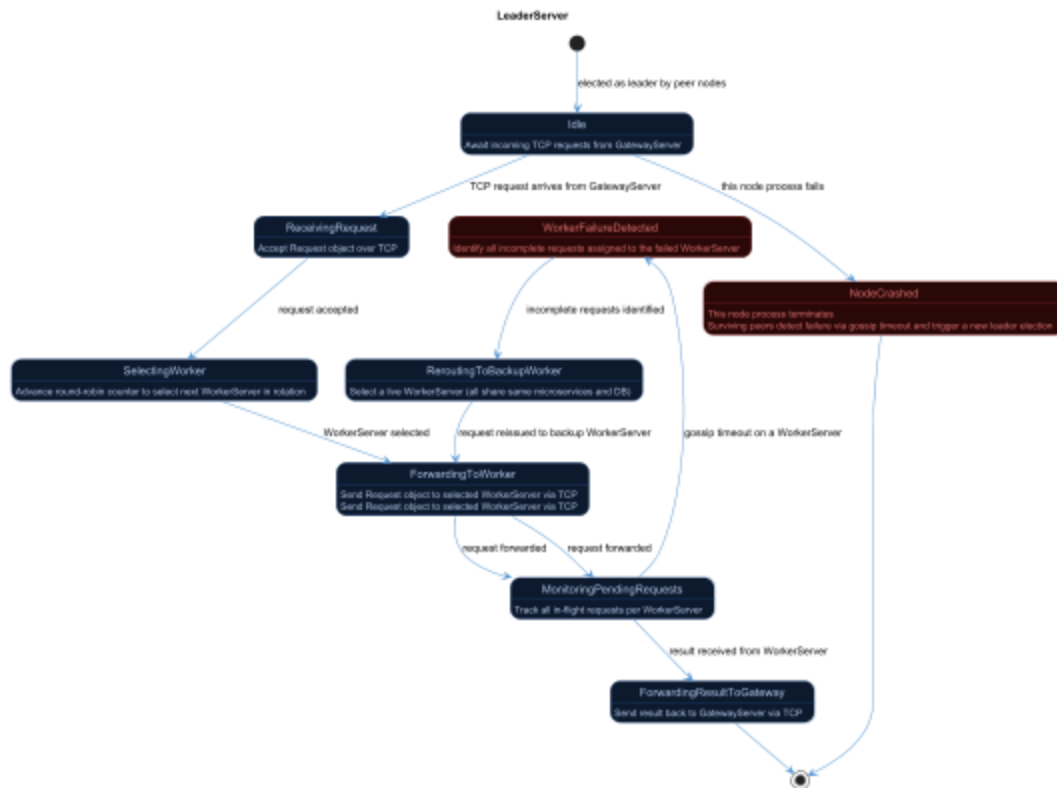


Diagram IV – WorkerServer



Diagram V – Microservice

This state diagram is of the PlayerHandler, which is one of the many microservices a WorkerServer delegates work to.

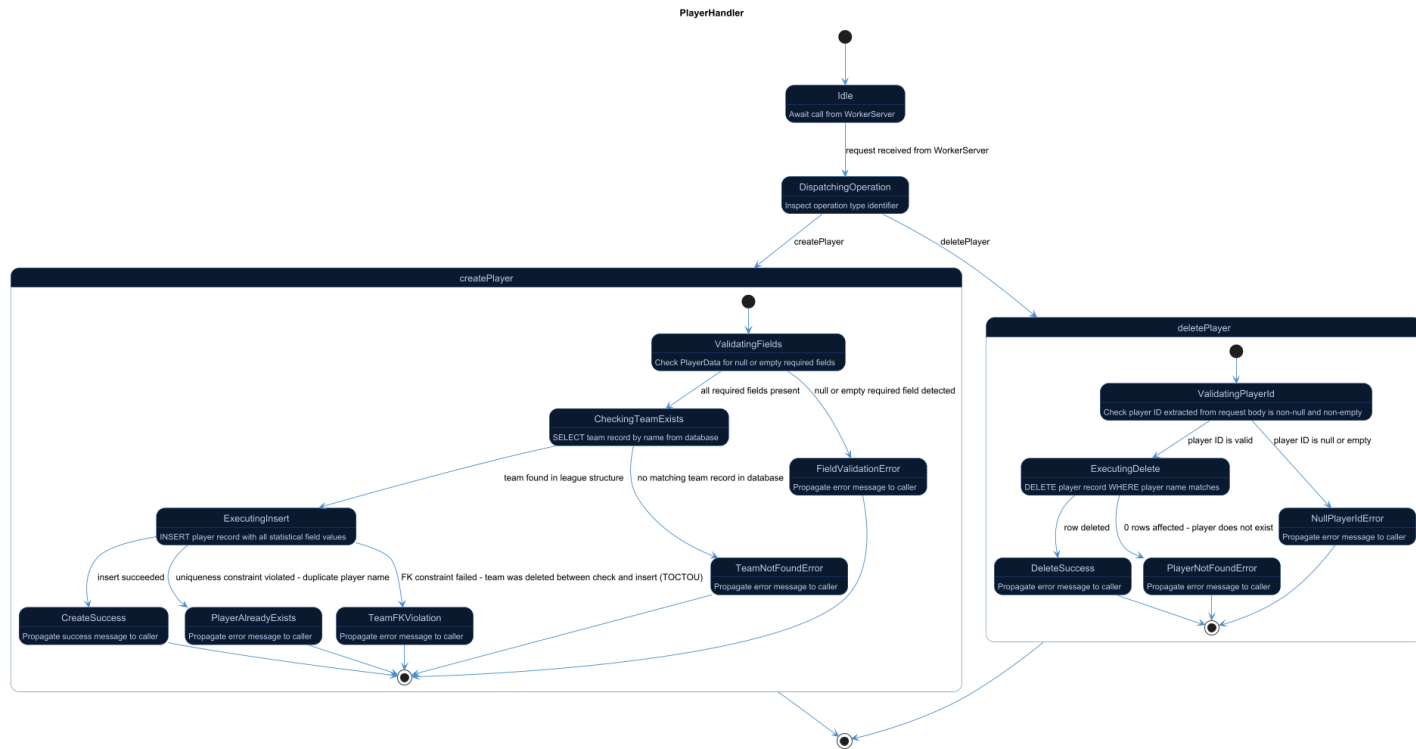


Diagram VI – PostgreSQL / Patroni Cluster

